BUILDING AI INFRASTRUCTURE · PART 3 OF 7

Part 3: AWS — EKS and GPUs

Chapter summary for beginners

In one sentence: build a private network, an EKS cluster and a GPU node group that starts at zero machines, then serve a large language model on a real GPU, and tear it all down the same day.

Video length: about 9 minutes · **Code:** nawex-platform, or this repo's `terraform/modules/eks` · **Cost:** a few dollars if you destroy the same day

The 8 blocks on AWS

#	Block	On AWS
1	Compute	EC2: general nodes + a GPU node group (**g5.xlarge**, 1× NVIDIA A10G)
2	Data and storage	**S3** for data and models; **ECR** for container images
3	Networking	VPC, private subnets, Application Load Balancer
4	Orchestration	**EKS**, managed Kubernetes
5	MLOps	MLflow + Kubeflow (or SageMaker)
6	Serving	KServe + **vLLM** on the GPU pool
7	Observability	CloudWatch + Prometheus + Grafana
8	Guardrails	IAM + **IRSA**, Secrets Manager, AWS Budgets

Three things before you build

1. **Budget alert first, always.** AWS Budgets, for example $10/month, with emails at 50%, 90% and 100%.
2. **GPU quota.** New accounts can't launch GPUs. Go to Service Quotas → EC2 → *Running On-Demand G and VT instances* and request at least 4 vCPUs. Approval can take days.
3. **Tools and login.** Install the AWS CLI, Terraform (1.6+), kubectl and helm, then check with `aws sts get-caller-identity`.

The big idea: a GPU pool that scales 0 → N → 0

The GPU node group has **min 0** and a **taint**, which is a "keep out" sign. Only pods that ask for a GPU (`nvidia.com/gpu`) can land there.

1. You deploy vLLM, and the pod asks for 1 GPU.
2. There's no GPU machine yet, so the pod waits in **Pending**.
3. The **cluster autoscaler** sees the pending pod and grows the pool from 0 to 1. A g5.xlarge starts.

4. The model loads, and a **readiness probe** holds traffic until it's ready.
5. You call it with an OpenAI-style API on `/v1`.
6. You delete the deployment, and after the scale-down delay the pool goes back to 0.

> **You only pay while the GPU is working.**

Build it

With nawex-platform (the path used in the video):

```
git clone https://github.com/Here2ServeU/nawex-platform && cd nawex-platform
cp deploy/terraform/live/aws/dev/terraform.tfvars.example
deploy/terraform/live/aws/dev/terraform.tfvars
GPU_ENABLED=1 scripts/deploy.sh aws dev plan     # always look before you build
GPU_ENABLED=1 scripts/deploy.sh aws dev apply    # about 15 minutes
kubectl get nodes -L nvidia.com/gpu.present -L node.kubernetes.io/instance-type
```

Or with this repo:

```
make tf-init ENV=dev
terraform -chdir=terraform/environments/dev plan -var gpu_enabled=true -out=plan.tfplan
make tf-apply ENV=dev
```

Turn on the GPU stack

Install the **NVIDIA GPU Operator** with `driver.enabled=false`, because the EKS NVIDIA image already includes the driver. The operator adds:

- **the device plugin**, which tells Kubernetes the node has a GPU;
- **the DCGM exporter**, which provides GPU metrics;
- **feature discovery**, which labels nodes with their GPU model.

Prove it works: the operator pods are Running, the node shows `nvidia.com/gpu: 1`, and `nvidia-smi` works in a test pod. In this repo, uncomment `infra-gpu` in `flux/clusters/dev-eks/infrastructure.yaml`.

Cost and clean-up

What costs money	How to stay near $0
EKS control plane (hourly)	Build, learn and destroy the same day
Every running node, GPUs most of all	GPU pool minimum 0
NAT gateway and load balancer	Budget alert on
Storage you leave behind	Check pricing for your region

Tear down: `scripts/deploy.sh aws dev destroy` (or `make tf-destroy ENV=dev`). Then check the console for leftover load balancers and volumes, which are the usual surprise on the bill.

Words to know

- **VPC:** your private network in AWS.
- **Node group:** a set of identical machines in an EKS cluster.
- **Taint / toleration:** a “keep out” sign on a node, and the pass a pod needs to get past it.
- **Cluster autoscaler:** adds nodes when pods are pending and removes idle ones.
- **IRSA:** IAM Roles for Service Accounts, which gives each pod its own AWS permissions with no keys.
- **vLLM:** a fast server for large language models.

Check yourself

1. Why does the vLLM pod start in Pending?
2. What does the taint on the GPU pool prevent?
3. Why `driver.enabled=false` on EKS?
4. What two things do people forget to delete?

Answers: 1) No GPU node exists until the autoscaler adds one. 2) Non-GPU work landing on expensive GPU machines. 3) The EKS NVIDIA image already includes the driver. 4) Load balancers and volumes (disks).

Next: Part 4, the same platform on Azure with AKS.

Emmanuel Naweji · emmanuelnaweji.com · transformed2succeed.com
Code: github.com/Here2ServeU/building-ai-infra · Series guide: docs/series/README.md